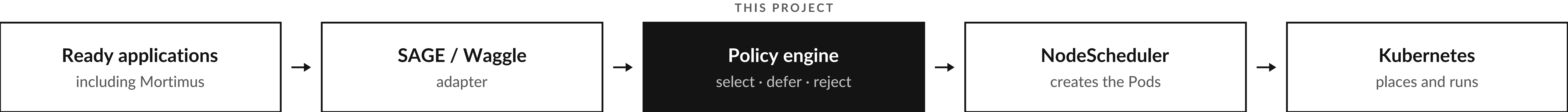


Resilient urgent orchestration for SAGE

An explainable scheduling policy for SAGE / Waggle nodes

Clément Nunes
Sage Grande Summer of AI 2026

In urgent computing, the right result delivered **too late** is the wrong result.



SAGE remains the orchestrator — only the selection policy changes. Beehive, WES, the NodeScheduler and Kubernetes are untouched.

The problem	How it decides	What it returns
SAGE nodes run several applications on limited edge hardware. Routine work and time-sensitive events compete for the same CPU, memory, GPU and execution slots, while priorities change. Three questions must be answered: - Which ready application should start next? - Can it start without exceeding safe limits? - Why was each application selected, deferred or rejected? Mortimus — cameras watching a landscape for smoke — is the integration target, not a dependency.	- Observe and validate ready and active tasks. - Rank valid tasks by priority, deadline slack, queue age and success probability. - Admit within concurrency limits, and resource limits when capacity is trustworthy. - Explain every selection, deferral or rejection with a stable reason. - Degrade to bounded queue order if it fails. URGENCY IS DEADLINE SLACK slack = deadline - now - estimated runtime	Every candidate leaves with one outcome and a stable reason. One node, one free slot, two tasks: smoke-detector SELECTED priority 90 · slack -5 s · score 80.75 reason: selected · predictedDeadlineMiss: true image-sampler DEFERRED priority 20 · slack +540 s · score 19.65 reason: concurrency_limit Deferred stays queued for the next free slot; rejected is invalid work.